

UCS503 - Software Engineering

Real-Time Text Simplification for Dyslexic Readers

Submitted by:

1024030174 Suneet Singh Arora
1024030175 Dhruv Kumar Garg

Group: 3C16 BE Third Year, CSE

Submitted to:

Paramveer Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	3
1.1	Background and Context	3
1.1.1	Problem Statement	3
1.2	Project Objectives	4
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	6
2.2	Implementation Details	6
2.2.1	Module 1: Core Functionality	6
2.2.2	Module 2: Secondary Features	7
3	Results and Evaluation	8
3.1	Testing Methodology	8
3.2	Performance Metrics	8
3.3	Analysis	9
4	Conclusion	10
4.1	Summary of Work	10
4.2	Key Findings	10
4.3	Future Work and Recommendations	10
4.4	Final Remarks	11

Introduction

1.1 Background and Context

Written text often contains difficult vocabulary and long sentences that make it hard to read, particularly for readers with dyslexia, limited reading proficiency, or non-native English speakers. Making text easier to read without losing its meaning is a well-known problem in Natural Language Processing, studied as text simplification.

Readability formulas such as the Flesch Reading Ease score, the Flesch–Kincaid Grade Level, and the SMOG Index estimate text difficulty from sentence length and syllable counts. These scores are proxies for difficulty rather than direct measures of comprehension, but they allow repeatable comparison between two versions of a text.

This project presents a rule based Python prototype that replaces selected difficult words and phrases with simpler alternatives, splits sentences where its rules allow, and compares readability statistics before and after simplification. It uses the `textstat` library and the NLTK CMU Pronouncing Dictionary, and does not use a trained machine learning model yet.

1.1.1 Problem Statement

Dyslexic readers often face challenges while processing complex written content due to difficulties with word recognition, sentence comprehension, and reading fluency. Existing accessibility tools may provide basic text adjustments but often lack intelligent simplification that preserves meaning while improving readability. The main problems are:

- **Complex text comprehension:** Long sentences, difficult vocabulary, and complex grammatical structures can increase cognitive load and make reading challenging for dyslexic individuals.
- **Limited text adaptation:** Existing solutions often provide formatting changes but do not dynamically simplify text based on linguistic complexity and reader needs.
- **Loss of meaning during simplification:** Basic text simplification approaches may reduce complexity but fail to preserve the original context and intent of the content.
- **Lack of real-time assistance:** Many accessibility tools require manual processing of text, limiting their usefulness for continuous reading experiences across digital platforms.

Why this matters now (contextual relevance): With increasing amounts of digital content being consumed daily, dyslexic readers need intelligent accessibility solutions that can reduce reading difficulty while preserving the original meaning of text. A real time AI based text simplification system can improve reading comprehension, provide personalized assistance, and create a more inclusive digital reading experience without replacing the reader's ability to engage with the original content.

1.2 Project Objectives

The objectives of this project are:

1. To implement a text processing pipeline in Python that accepts English text and prepares it for analysis.
2. To simplify selected difficult words and phrases using a manually defined set of replacement and sentence splitting rules.
3. To calculate standard readability measures (Flesch Reading Ease, Flesch–Kincaid Grade Level, and SMOG Index) along with word count, sentence count, average sentence length, and difficult word count.
4. To compare the original and simplified text and summarize the change in each metric.
5. To test the prototype on a small set of varied texts and document its limitations, including cases where substitutions reduce grammatical quality or alter meaning.

Methodology and Design

2.1 System Architecture

The system is a modular command line pipeline written in Python, with each stage implemented as a separate module.

2.1.1 High-Level Design

The input text is first cleaned by the preprocessing module. The readability module then computes statistics for the original text. The rule-based simplifier applies word/phrase replacements and sentence splitting, after which the readability module is run again on the simplified text. Finally, the evaluation module compares the two sets of statistics, and `main.py`, which orchestrates all stages, prints the results to the terminal.

Module	Responsibility
<code>preprocessing.py</code>	Cleans the input text before processing.
<code>readability.py</code>	Computes word count, sentence count, average sentence length, Flesch Reading Ease, Flesch–Kincaid Grade Level, SMOG Index, and difficult words.
<code>simplifier.py</code>	Applies rule-based replacements and sentence splitting.
<code>evaluation.py</code>	Compares original and simplified statistics.
<code>main.py</code>	Connects the modules and prints results.

Table 2.1: Modules and their responsibilities

2.1.2 Technical Stack

- **Framework/Language:** Python, run as a command line application inside a virtual environment (`.venv`).
- **Database:** None; the prototype processes text in memory and does not require persistent storage.
- **Tools:** Visual Studio Code, terminal, Git, and GitHub.
- **Libraries:** `textstat` for readability scoring and NLTK with the CMU Pronouncing Dictionary (`cmudict`) for syllable counting.

2.2 Implementation Details

The prototype is implemented in Python as a set of small modules, each handling one stage of the pipeline.

2.2.1 Module 1: Core Functionality

The core module is the rule based simplifier (`simplifier.py`). It applies two kinds of manually defined rules to the input text:

- **Word and phrase substitution:** selected difficult words and phrases are replaced with simpler alternatives. For example, “makes use of” becomes “uses” and “a large number of” becomes “many”.
- **Sentence splitting:** sentences are divided at points defined by the splitting rules, which shortens average sentence length.

The rules are context insensitive, so a replacement may be unsuitable in some sentences and can produce awkward wording or alter meaning. The module therefore makes no claim of general semantic understanding.

Illustrative example (single run, not an overall result):

- **Input:** The system makes use of numerous computational resources to process a large number of files.
- **Output:** The system uses a lot of computing power to process many files.

2.2.2 Module 2: Secondary Features

The remaining modules support and evaluate the simplifier. Together they prepare the input, measure how difficult the text is, compare the results, and connect the stages into a single workflow.

- **Preprocessing** (`preprocessing.py`): This module prepares the raw input string before it is analysed or simplified. Cleaning the text first ensures that the later stages receive a consistent input, so that the readability calculations and the replacement rules are not affected by irregularities in the original text.
- **Readability analysis** (`readability.py`): uses `textstat` to compute word count, sentence count, average sentence length, Flesch Reading Ease, Flesch–Kincaid Grade Level, SMOG Index, and the count and list of difficult words. Syllable counts rely on the NLTK CMU Pronouncing Dictionary.
- **Evaluation** (`evaluation.py`): This module takes the readability statistics of the original and simplified text and computes the change in each metric. It then produces an improvement summary indicating whether the simplified text scores as easier to read. Because readability scores are only proxies for difficulty, this summary reflects changes in the scores and does not confirm that meaning or grammatical quality has been preserved.
- **Orchestration** (`main.py`): connects all modules and prints the results to the terminal.

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Each module (`preprocessing.py`, `readability.py`, `simplifier.py`, `evaluation.py`) is to be checked individually on sample inputs. Automated unit tests are planned and have not yet been implemented.
- **Integration Testing:** The full pipeline was run end to end through `main.py` on sample text, and the printed before/after comparison was inspected manually.
- **Performance Testing:** Not yet carried out. No response time or throughput measurements have been recorded.
- **User Testing:** Not carried out. No user study has been conducted.

3.2 Performance Metrics

Readability is measured using word count, sentence count, average sentence length, Flesch Reading Ease, Flesch–Kincaid Grade Level, SMOG Index, and difficult word count. Table 3.2 shows one illustrative run; it is a single example, not an average or proof of overall performance.

Version	Text
Original	The system makes use of numerous computational resources to process a large number of files.
Simplified	The system uses a lot of computing power to process many files.

Table 3.1: Sample input text and its simplified output

Metric	Original	Simplified
Word count	15	12
Flesch Reading Ease	44.97	60.71
Flesch–Kincaid Grade Level	10.71	7.77
SMOG Index	13.02	8.84
Difficult-word count	3	1

Table 3.2: Readability metrics for the sample text in Table 3.1

3.3 Analysis

- **Did you meet your objectives?** The pipeline, rule based simplifier, readability calculation, and before/after comparison are implemented and run locally. Testing across a varied set of texts is still pending, so the objective of documenting limitations is only partly met.
- **Where did you exceed expectations?** In the illustrative run, substitution and sentence splitting lowered the grade level scores and the difficult word count.
- **What challenges did you encounter?** Rule-based replacements ignore context and can produce awkward wording or alter meaning. Readability scores are only proxies for difficulty, and sentence splitting changes sentence counts and average sentence length, which also affects the scores.
- **How did you resolve them?** These issues are not yet resolved; they are listed as future work: larger curated rules, context aware replacement, meaning preservation checks, and automated tests.

Conclusion

4.1 Summary of Work

This project implemented a rule based text simplification prototype in Python. The system accepts English text, applies manually defined word/phrase replacements and sentence splitting rules, computes readability statistics for the original and simplified versions, and prints a comparison in the terminal. The objectives of building the pipeline, simplifying selected wording, calculating standard readability measures, and comparing original and simplified text were met. The objective of testing on a varied set of texts and documenting limitations is only partly met, as broader evaluation is still pending.

4.2 Key Findings

- **Finding 1:** In the illustrative test run, substitution and sentence splitting improved the readability scores and reduced the difficult word count.
- **Finding 2:** Readability scores are only proxies for difficulty, so a better score does not guarantee preserved meaning or grammatical quality.
- **Finding 3:** Context-insensitive rules can produce awkward or unsuitable replacements.

4.3 Future Work and Recommendations

1. **Potential enhancements:** larger and better curated replacement rules, more robust sentence handling, and automated unit and integration tests.
2. **Alternative approaches:** context aware word replacement and, later, trained simplification models compared against the rule based baseline.
3. **Scalability improvements:** support for file input and batch processing of larger texts, and a larger evaluation dataset.
4. **Integration with other systems:** an optional graphical or web interface for non-technical users.
5. **Real-world deployment considerations:** meaning preservation checks and user testing, including with dyslexic readers, before any practical use.

4.4 Final Remarks

The prototype shows that simple, transparent rules can lower readability scores on suitable text, while also showing the limits of rule based simplification without context or meaning checks. It provides a foundation that can be extended with broader evaluation and more capable methods.